LifeQuest

Gamified Habit Tracker

Complete Project Documentation

For coders **and** non-technical readers

Full-stack web application
React (Vite) · Express.js · MongoDB · JWT

Generated documentation guide · Inside LifeQuest folder
File: `LifeQuest-Documentation.pdf`

Table of Contents

1. What is LifeQuest? (Non-technical overview)
2. How to explain this project (presentation script)
3. Architecture & how the pieces talk
4. Tech stack & setup
5. Folder & file map
6. Features ↔ code map
7. Backend — line-by-line explanations
8. Frontend — line-by-line explanations
9. Gamification rules (XP, levels, badges, streaks)
10. API reference (all endpoints)
11. Database models summary
12. Demo accounts & environment
13. Glossary (tech words in plain English)

1. What is LifeQuest? (Non-technical overview)

LifeQuest turns your daily life into a game. Instead of only writing a to-do list, you track habits, earn points (XP), level up, unlock badges, finish daily missions, and compete on a leaderboard — like a role-playing game for real habits.

Real life	In LifeQuest
Good habits (exercise, study, read)	Habits you create and mark complete
Getting better over time	XP (experience points) and levels
Rewards for consistency	Coins and badges
Daily goals	Missions
Comparing with friends	Leaderboard
To-do list	Tasks (daily / weekly / monthly)
Diary + how you feel	Journal with mood tracker
Reminders and achievements	Notifications

> **Simple user journey:**
> Sign up → Create habits → Mark them done each day → Earn XP → Level up → Unlock badges → Complete missions → Climb the leaderboard.

Three main parts

1. **Frontend (client)** — the website you see in the browser (usually port 5173).
2. **Backend (server)** — the "brain" that checks login, saves data, awards XP (port 5000).
3. **Database (MongoDB)** — permanent storage for users, habits, tasks, journals, etc.

2. How to explain this project (presentation script)

Use this when presenting to teachers, interviewers, or non-tech audiences:

> "LifeQuest is a **full-stack gamified habit tracker**. Users track habits, complete daily missions, and earn XP, coins, and badges — like an RPG for real life.
>
> The **frontend** is React with Vite: pages for dashboard, habits, calendar, analytics, tasks, journal, notifications, settings, and an admin panel.
>
> The **backend** is Express with MongoDB: JWT login, REST APIs for every feature, and gamification logic for streaks, levels, and badges.
>
> When you complete a habit, the server updates the database, awards XP, may level you up, updates your streak, and can unlock achievements."

One-sentence pitch

LifeQuest makes building good habits fun by turning progress into levels, rewards, and friendly competition.

3. Architecture & how the pieces talk

```
Browser (React app on port 5173)
        │  JSON requests (Axios) + JWT token
        ▼
Express API (server/index.js on port 5000)
        │  middleware/auth.js checks the token
        ▼
Routes  →  Controllers (business rules)
                │
                ▼
           Models (Mongoose schemas)
                │
                ▼
           MongoDB (lifequest database)
```

Request example (plain English)

1. User clicks "Complete habit" on the Habits page.
2. Frontend calls `POST /api/habits/:id/complete` with the login token.
3. `protect` middleware verifies the token and loads the user.
4. `completeHabit` controller updates the habit, XP, streak, progress, badges.
5. Server returns new stats; UI shows XP / level-up overlay.

4. Tech stack & setup

Layer	Technologies
Frontend	React 19, Vite, React Router, Axios, Framer Motion, Recharts, Lucide icons
Backend	Node.js, Express 5, Mongoose, bcryptjs, jsonwebtoken, cors, dotenv
Database	MongoDB
Auth	JWT (JSON Web Token) stored in browser localStorage

How to run

```
# From LifeQuest root
npm run install:all

# Seed demo users
npm run seed

# Terminal 1 — API
npm run dev:server     →  http://localhost:5000

# Terminal 2 — UI
npm run dev:client     →  http://localhost:5173
```

5. Folder & file map

Path	Plain English purpose
`client/`	Website UI (React SPA)
`client/src/pages/`	Full screens: Dashboard, Habits, Login…
`client/src/components/`	Reusable UI: modal, loader, habit card, layout
`client/src/context/`	Shared state: logged-in user, dark/light theme
`client/src/api/`	Functions that call the backend
`client/src/styles/`	Global look & feel (glassmorphism theme)
`server/`	API server
`server/routes/`	URL → which function runs
`server/controllers/`	Actual business logic
`server/models/`	Shape of data saved in MongoDB
`server/middleware/`	Security checks before requests
`server/utils/`	Helpers: tokens, badges, streaks
`documentation/`	This guide’s source HTML + PDF generator (not app runtime code)

6. Features ↔ code map

Feature	Frontend	Backend
Login / Register / Reset	pages/auth/*, AuthContext	authRoutes, authController
Dashboard	pages/Dashboard.jsx	progress/dashboard
Habits CRUD + complete	Habits.jsx, HabitCard, HabitFormModal	habitRoutes, habitController
Calendar	pages/Calendar.jsx	progress/calendar
Analytics charts	pages/Analytics.jsx	progress/analytics
Missions	pages/Missions.jsx	missionRoutes
Leaderboard + friends	pages/Leaderboard.jsx	leaderboardRoutes
Tasks	pages/Tasks.jsx	taskRoutes, taskController
Journal + mood	pages/Journal.jsx	journalRoutes
Notifications	pages/Notifications.jsx	notificationRoutes
Settings / theme	Settings.jsx, ThemeContext	auth profile endpoints
Admin panel	pages/Admin.jsx	adminRoutes + admin middleware

7. Backend — line-by-line explanations

7.1 server/index.js — the server starts here

Line(s)	Code idea	What it means
1–13	`import express, cors, routes…`	Load libraries and every feature's route file
15	`dotenv.config()`	Read secrets from `server/.env` (PORT, MongoDB, JWT)
16	`connectDB()`	Open connection to MongoDB
18	`const app = express()`	Create the web application object
20	`app.use(cors(…))`	Allow the React app (port 5173) to call this API
21	`express.json({ limit: '2mb' })`	Accept JSON request bodies up to 2 MB
23	`GET /api/health`	Health check: "Is the API alive?"
25–33	`app.use('/api/…')`	Mount auth, habits, tasks, progress, journal, notifications, missions, leaderboard, admin
35–38	`error middleware`	Catch crashes → return HTTP 500 + message
40–41	`app.listen(PORT)`	Start listening (default **5000**)

7.2 server/config/db.js — database connection

Line(s)	Code idea	What it means
1–2	`import mongoose, dotenv`	MongoDB ODM + env loader
4	`dotenv.config()`	Load MONGODB_URI
6–14	`connectDB async`	Connect; log host on success; exit process on failure
16	`export default`	So index.js can call connectDB()

7.3 server/middleware/auth.js — security gate

> Analogy: `protect` is the bouncer checking your wristband. `admin` is VIP-only access.

Line(s)	Code idea	What it means
4–8	`protect reads Bearer token`	Look for `Authorization: Bearer <token>`
10–12	`no token → 401`	Not logged in
15–16	`jwt.verify + find user`	Validate token; load user without password field
17–19	`inactive / missing`	Reject deactivated accounts
20	`next()`	Allow the request to continue
21–23	`catch`	Bad or expired token → 401
26–29	`admin`	Only role === 'admin' may pass; else 403

7.4 server/routes/taskRoutes.js — example of routing

(This is the small file that maps URLs to task functions.)

Line	Code idea	What it means
1	`import express`	Need Express to create a router
2	`import getTasks, createTask…`	Bring in controller functions
3	`import protect`	Require login for every task call
5	`const router = …`	Create a mini URL map
6	`router.use(protect)`	All routes below need a valid JWT
7	`GET / and POST /`	List tasks / create task
8	`PUT /:id and DELETE /:id`	Update or delete one task by id
10	`export default`	Mounted in index.js at `/api/tasks`

7.5 Other route files (same idea)

File	Base URL	Job
authRoutes.js	/api/auth	Register, login, profile, password, export, delete account
habitRoutes.js	/api/habits	CRUD + POST /:id/complete
progressRoutes.js	/api/progress	Dashboard, calendar, day detail, analytics
journalRoutes.js	/api/journal	List, by date, upsert, delete
notificationRoutes.js	/api/notifications	Inbox, mark read, generate daily
missionRoutes.js	/api/missions	List & complete daily missions
leaderboardRoutes.js	/api/leaderboard	Global / friends boards, add friend
adminRoutes.js	/api/admin	Admin-only users, analytics, badges, reports

7.6 server/models/User.js — the player profile

Concept	Fields / methods	Plain English
Identity	name, email, password	Who the person is
Role	user / admin	Normal user vs administrator
Game stats	xp, level, coins	Progress and rewards
Streaks	currentStreak, longestStreak, lastActiveDate	How consistently they show up
Social / rewards	badges, friends	Achievements and friends list
Settings	theme, isActive	Dark/light + account enabled
Password reset	resetPasswordToken, resetPasswordExpire	Forgot-password flow
pre('save')	bcrypt.hash	Never store plain-text passwords
matchPassword	bcrypt.compare	Check password at login
getXpForLevel / addXp	level math	When XP crosses threshold → level++ and +50 coins

7.7 server/models/Habit.js

- Belongs to a **user** (foreign key).
- **category**: Health, Fitness, Study, Coding, Finance, Meditation, Reading, Business, Personal.
- **difficulty**: Easy / Medium / Hard → XP 10 / 20 / 40.

- **target**: times per day; **completions**: list of YYYY-MM-DD dates done.
- Tracks **currentStreak**, **longestStreak**, **totalCompletions**.
- `getXpReward()` returns XP based on difficulty.

7.8 server/models/Task.js

- `type`: daily | weekly | monthly
- `priority`: low | medium | high
- `status`: todo | in-progress | completed
- Optional `deadline` and `notes`

7.9 Other models

Model	Job
Journal	One diary entry per user per day + mood emoji + tags (unique user+date)
Mission	Daily quest with coinReward / xpReward / completed flag
Notification	Inbox: reminder, motivation, achievement, missed, system
Progress	Daily snapshot for charts: completionRate, xpEarned, coinsEarned

7.10 Controllers — what each file does

Controller	Responsibility
authController.js	Register, login, getMe (+ quote), update profile, change/forgot/reset password, delete account, export data
habitController.js	List/create/update/delete habits; **completeHabit** awards XP, updates streaks, progress, badges
taskController.js	Filter list by type/status; create/update/delete own tasks only
progressController.js	Dashboard stats, calendar heatmap data, day detail, analytics aggregates
journalController.js	List, get-by-date, upsert, delete journal entries
notificationController.js	List, mark read, mark all, delete, generate daily notifications
missionController.js	Ensure daily missions exist; complete for rewards
leaderboardController.js	Global & friends rankings; add friend by email
adminController.js	Manage users, platform analytics, badge overview, reports

7.11 taskController.js — line meaning

Lines	Function	What it means
3–13	`getTasks`	Find this user's tasks; optional filters type/status; sort by deadline
15–23	`createTask`	Create task owned by logged-in user; return 201
25–37	`updateTask`	Update only if id + user match; 404 if missing
39–46	`deleteTask`	Delete only own task; confirm message

7.12 habitController completeHabit (core game loop)

1. Find the habit belonging to the user.
2. If already completed today → uncomplete (remove XP/streak effects).
3. Else add today's completion, increase totals, call `updateUserStreak`.
4. Award XP via `user.addXp`; may level up and grant coins.
5. Update daily Progress snapshot.
6. Run `checkAndUnlockBadges`.
7. Return habit + rewards so UI can celebrate.

7.13 utils/helpers.js & gamification.js

- **generateToken(id)** — create JWT for login sessions.
- **todayStr / formatDate / daysBetween** — date helpers (YYYY-MM-DD).
- **BADGE_DEFS / MISSION_TEMPLATES / MOTIVATIONAL_QUOTES** — content definitions.
- **checkAndUnlockBadges** — streak & category milestones → badge + notification.
- **updateUserStreak** — if active yesterday, streak++; else reset to 1.

7.14 server/seed.js

Creates starter accounts so demos work immediately:

- Admin: `admin@lifequest.app` / `admin123`
- Demo: `demo@lifequest.app` / `demo123`

8. Frontend — line-by-line explanations

8.1 client/src/main.jsx — React starts here

Line(s)	Code idea	What it means
1–4	`imports`	React, App component, global CSS
6–10	`createRoot(#root).render`	Put the app into the HTML page's root div (StrictMode helps catch issues)

8.2 client/src/App.jsx — page map (router)

Part	What it means
Imports	All pages + AuthProvider + ThemeProvider + guards
PublicOnly	If already logged in, don't show login/register — go to dashboard
/login, /register, /forgot-password	Public auth screens
ProtectedRoute + AppLayout	Must be logged in; shows sidebar shell
Inner routes	dashboard, habits, calendar, analytics, missions, leaderboard, tasks, journal, notifications, settings
/admin + adminOnly	Administrators only
/ and *	Default / unknown URLs → dashboard

8.3 client/src/api/client.js

Line(s)	What it means
3	`Read VITE_API_URL (fallback localhost:5000/api)`
5–8	`Create Axios instance with JSON headers`
10–14	`Request interceptor: attach JWT from localStorage key` `lq_token`
16–28	`If API returns 401, clear login and redirect to /login`

8.4 client/src/api/index.js

Named helpers so pages stay clean:

- `authAPI`, `habitAPI`, `taskAPI`, `progressAPI`
- `journalAPI`, `notificationAPI`, `missionAPI`
- `leaderboardAPI`, `adminAPI`

8.5 AuthContext.jsx — "Who am I?"

Piece	Plain English
user state from localStorage	Stay logged in after page refresh
showToast	Temporary success/error banner (~3.2s)
persist(token, user)	Save token + user after login/register
refreshUser	Call /auth/me to refresh profile + quote
login / register / logout	Auth actions used by pages
isAdmin	user.role === 'admin'

8.6 ThemeContext.jsx

- Reads theme from localStorage or user profile.
- Sets `data-theme` on <html> so CSS switches dark/light.
- toggleTheme also saves preference to the server when logged in.

8.7 ProtectedRoute.jsx

1. Still loading → show Loader.
2. No user → redirect to /login.
3. adminOnly but not admin → /dashboard.
4. Else render child routes via <Outlet />.

8.8 Common components

Component	Purpose
Loader.jsx	Full-page spinner with optional text
Modal.jsx	Reusable dialog wrapper
Toast.jsx	Shows AuthContext toast messages
ProgressRing.jsx	Circular SVG progress for habit completion
LevelUpOverlay.jsx	Celebration animation when user levels up

8.9 Habit UI

- **HabitCard** — shows one habit, complete/edit/delete, progress ring.
- **HabitFormModal** — create/edit form (name, category, difficulty, target, reminder).

8.10 AppLayout.jsx

Main shell: sidebar navigation, header with user stats, theme toggle, logout, and the page content area.

8.11 Pages (screens)

Page	What the user sees / does
Dashboard	Level, XP, streak, coins, daily/monthly progress, quote
Habits	Manage and complete habits
Calendar	Month view with completion % and day details
Analytics	Charts (Recharts): trends, pie, heatmap
Missions	Daily quests with XP/coin rewards
Leaderboard	Global & friends rankings
Tasks	Daily/weekly/monthly task board
Journal	Diary + mood + tags
Notifications	Inbox read/unread/delete
Settings	Profile, theme, password, export, delete account
Admin	Users, analytics, badges, reports
Login / Register / ForgotPassword	Public authentication flows

8.12 utils/constants.js

Shared lists: categories, difficulties, XP map, category colors, moods, and date format helpers used across the UI.

8.13 styles/global.css

Large design-system stylesheet (~1000+ lines): CSS variables for dark/light themes, glassmorphism panels, layout, buttons, cards used for interactive forms, sidebar, charts wrappers. Non-tech view: this file is the “look and feel” of the whole app.

9. Gamification rules

XP by difficulty

Difficulty	XP
Easy	10
Medium	20
Hard	40

Levels

XP needed for level L uses roughly: `floor(100 × L^1.5)` (cumulative threshold). Crossing a new level grants **+50 coins**.

Streaks

- Complete something on consecutive calendar days → streak grows.
- Miss a day → streak resets to 1 on next activity.
- Longest streak is remembered separately.

Badges (examples)

Badge	Unlock idea
7 / 30 / 100 Day Streak	Maintain streak milestones
Early Bird	Complete a habit before 8 AM
Book Worm / Gym Hero / Coding Master / Meditation Monk	10+ completions in that category

10. API reference (all endpoints)

Base URL: `http://localhost:5000/api`. Protected routes need header: `Authorization: Bearer <token>`.

Auth

Method	Path	Auth?	Purpose
POST	/auth/register	No	Create account
POST	/auth/login	No	Login → token
POST	/auth/forgot-password	No	Start reset
POST	/auth/reset-password	No	Finish reset
GET	/auth/me	Yes	Current user + quote
PUT	/auth/profile	Yes	Update profile/theme
PUT	/auth/change-password	Yes	Change password
DELETE	/auth/account	Yes	Delete account
GET	/auth/export	Yes	Export user data

Habits

Method	Path	Purpose
GET	/habits	List habits (+ completedToday)
POST	/habits	Create
GET/PUT/DELETE	/habits/:id	Read / update / delete
POST	/habits/:id/complete	Toggle today’s completion + rewards

Tasks

Method	Path	Purpose
GET	/tasks	List (optional ?type & ?status)
POST	/tasks	Create
PUT	/tasks/:id	Update
DELETE	/tasks/:id	Delete

Progress

Method	Path	Purpose
GET	/progress/dashboard	Home stats
GET	/progress/calendar	Month data (?year & month)
GET	/progress/calendar/:date	One day detail
GET	/progress/analytics	Chart aggregates

Journal / Notifications / Missions / Leaderboard / Admin

- **Journal:** GET /, GET /:date, POST / (upsert), DELETE /:id
- **Notifications:** GET /, PUT /:id/read, PUT /read-all, DELETE /:id, POST /generate-daily
- **Missions:** GET /, POST /:id/complete
- **Leaderboard:** GET /global, GET /friends, POST /friends { email }
- **Admin:** users list/toggle/delete, analytics, badges, reports (admin role)
- **Health:** GET /health → { status: ok }

11. Database models summary

```
User ──< Habit
  │
  ├──< Task
  ├──< Journal (unique per date)
  ├──< Mission (unique per date+title)
  ├──< Notification
  ├──< Progress (unique per date)
  └──  friends[] → other Users
```

All user-owned documents store `user: ObjectId` so one person cannot edit another person’s data.

12. Demo accounts & environment

Demo accounts (after `npm run seed`)

Role	Email	Password
Admin	admin@lifequest.app	admin123
Demo	demo@lifequest.app	demo123

server/.env

```
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/lifequest
JWT_SECRET=your_secret
JWT_EXPIRE=7d
CLIENT_URL=http://localhost:5173
```

client/.env

```
VITE_API_URL=http://localhost:5000/api
```

13. Glossary (tech words in plain English)

Term	Simple meaning
Frontend	What users see and click
Backend / API	Server that stores data and enforces rules
Database	Long-term storage (MongoDB here)
JWT	Digital pass proving you logged in
Route	A URL path that triggers a function
Controller	Code that performs the action for a route
Model / Schema	Blueprint for one type of saved record
Middleware	Code that runs before the main action (e.g. auth check)
CRUD	Create, Read, Update, Delete
SPA	Single Page App — one site that changes screens without full reloads
XP / Level / Badge	Game-style progress, rank, and achievement
Streak	Number of days in a row you stayed active
REST	Common style of API using GET/POST/PUT/DELETE

> **Where this PDF lives:** at the LifeQuest folder root as `LifeQuest-Documentation.pdf`, separate from runtime app code in `client/` and `server/`. Source HTML and generator sit in `documentation/` (not required to run the app).

End of LifeQuest Complete Project Documentation. Use this PDF to study, present, or onboard others.